Lane2 Language Specification

Version: v1 draft
Status: working specification

Contents

1 Introduction

Lane2 is a strict, pure, expression-oriented functional programming language. Its first implementation target is a parser, a type checker, and an AST interpreter; later implementations may add bytecode compilation, a virtual machine, a linker, and effect handling.

Lane2 is intentionally small. It has no mutable state, assignment, trait system, method dispatch, module system, or implicit typeclass-style instance search in v1. Instead, v1 is centered on nominal data, first-class functions, bidirectional local type inference, exhaustive pattern matching, and explicit operation values opened into lexical scope.

This document specifies Lane2/Core v1: the platform-independent part of Lane2 that should behave the same across the initial AST interpreter and later execution backends.

1.1 Scope

Lane2/Core v1 covers:

- source structure and declarations;
- lexical scope and name resolution;
- nominal struct and enum types;
- function and block expressions;
- local type inference;
- pattern matching;
- `open`, `preopen`, and operation-based operators;
- unsafe builtin expressions.

The following are outside Lane2/Core v1:

- mutation and assignment;
- modules and imports;
- bytecode and virtual machine semantics;
- linker entrypoint selection;
- algebraic effects;
- type aliases;
- traits, typeclasses, and interfaces;
- tuples and collection syntax.

1.2 Compatibility

This specification is a v1 draft. No compatibility is promised between draft revisions. Language rules may be added, removed, or changed while the first implementation is being developed.

Once a v1 implementation is declared stable, this section should be replaced by an explicit compatibility policy.

1.3 Experimental Features

This draft does not distinguish stable and experimental language features. Every rule in this document is provisional until v1 is stabilized.

Future drafts may mark individual features as experimental when their surface syntax or semantics are intentionally unsettled.

1.4 Feedback

Design feedback is tracked in the Lane2 repository. Implementation changes should update this specification, `CONTEXT.md`, and ADRs when the change affects user-visible semantics.

1.5 Reference

When referencing this draft, use:

> Lane2 Language Specification : Lane2/Core v1 draft.

2 Syntax and Grammar

2.1 Notation

```
grammar        ::= { production }
production     ::= nonTerminal "::=" grammarExpression
grammarExpression ::=
    grammarSequence { "|" grammarSequence }
grammarSequence ::=
    { grammarItem }
grammarItem ::=
    terminal
  | nonTerminal
  | "[" grammarExpression "]"
  | "{" grammarExpression "}"
  | "(" grammarExpression ")"
  | grammarItem "?"
  | grammarItem "*"
  | grammarItem "+"

terminal       ::= "'" terminalText "'"
nonTerminal    ::= lowerCamelIdentifier

empty sequence denotes epsilon.
"A B" denotes sequencing.
"A | B" denotes choice.
"[ A ]" and "A?" denote optional occurrence.
"{ A }" and "A*" denote zero or more occurrences.
"A+" denotes one or more occurrences.
Parenthesized grammar expressions group without producing syntax.
```

2.2 Lexical Grammar

```
lexicalInput ::=
    (trivia | token)* eof

trivia ::=
    whitespace
  | lineTerminator
  | comment

token ::=
    keyword
  | reservedWord
  | identifier
  | intLiteral
  | stringLiteral
  | boolLiteral
  | operatorToken
  | punctuationToken

keyword ::=
    "struct"
  | "enum"
  | "fn"
  | "let"
```

```
  | "open"
  | "if"
  | "else"
  | "match"
  | "builtin"

reservedWord ::=
    "effect"
  | "handler"
  | "module"
  | "import"
  | "pub"
  | "type"
  | "return"

identifier ::=
    asciiIdentifierStart identifierContinue*
  | "_" identifierContinue+

identifierContinue ::=
    "_"
  | asciiIdentifierStart
  | decimalDigit

boolLiteral ::=
    "true"
  | "false"

intLiteral ::=
    decimalDigit+

stringLiteral ::=
    "\"" stringElement* "\""

stringElement ::=
    asciiStringCharacter
  | stringEscape

asciiStringCharacter ::=
    asciiCodePointExceptControlBackslashQuote

stringEscape ::=
    "\\" "\\"
  | "\\" "\""
  | "\\" "n"
  | "\\" "r"
  | "\\" "t"
  | "\\" "x" asciiHexByte

asciiHexByte ::=
    asciiHexLowByte
  | asciiHexHighByte

asciiHexLowByte ::=
    ("0" | "1" | "2" | "3" | "4" | "5" | "6") asciiHexDigit

asciiHexHighByte ::=
    "7" ("0" | "1" | "2" | "3" | "4" | "5" | "6" | "7")

operatorToken ::=
    "+"
```

```
  | "-"
  | "*"
  | "/"
  | "%"
  | "=="
  | "!="
  | "<"
  | "<="
  | ">"
  | ">="
  | "&&"
  | "||"
  | "!"
  | "|>"

punctuationToken ::=
    "("
  | ")"
  | "{"
  | "}"
  | "["
  | "]"
  | ","
  | "."
  | ":"
  | "::"
  | "->"
  | "=>"
  | "="
  | "_"

whitespace ::=
    U+0009
  | U+000B
  | U+000C
  | U+0020

lineTerminator ::=
    U+000A
  | U+000D
  | U+000D U+000A

decimalDigit ::=
    "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

asciiHexDigit ::=
    decimalDigit
  | "a" | "b" | "c" | "d" | "e" | "f"
  | "A" | "B" | "C" | "D" | "E" | "F"

asciiIdentifierStart ::=
    asciiUppercaseLetter
  | asciiLowercaseLetter

asciiUppercaseLetter ::=
    any ASCII code point from U+0041 through U+005A

asciiLowercaseLetter ::=
    any ASCII code point from U+0061 through U+007A
```

Lexical analysis uses maximal munch. If more than one token class matches the same maximal source span, keyword, reservedWord, and boolLiteral take priority over identifier.

2.3 Syntax Grammar

```
sourceFile ::=
    topLevelDeclaration* eof

topLevelDeclaration ::=
    structDeclaration
  | enumDeclaration
  | functionDeclaration
  | topLevelLetDeclaration
  | topLevelOpenLetDeclaration
  | openDeclaration

structDeclaration ::=
    "struct" typeName typeParameters? "{" structMember* "}"

structMember ::=
    fieldDeclaration
  | fieldForwardingDeclaration

fieldDeclaration ::=
    fieldName space ":" space type

fieldForwardingDeclaration ::=
    "open" valueName

enumDeclaration ::=
    "enum" typeName typeParameters? "{" enumVariant* "}"

enumVariant ::=
    variantName enumPayload?

enumPayload ::=
    "(" commaSeparatedTypes? ")"

functionDeclaration ::=
    "fn" typeParameters? functionName "(" commaSeparatedParameters? ")" "->" type block

parameter ::=
    valueName space ":" space type

topLevelLetDeclaration ::=
    "let" valueName space ":" space type "=" expression

topLevelOpenLetDeclaration ::=
    "let" "open" valueName space ":" space type "=" expression

localLetDeclaration ::=
    "let" valueName typeAnnotation? "=" expression

localOpenLetDeclaration ::=
    "let" "open" valueName typeAnnotation? "=" expression

typeAnnotation ::=
    space ":" space type

openDeclaration ::=
    "open" valueName
```

```
type ::=
    typeConstructor typeArguments?
  | functionType

typeConstructor ::=
    typeName

typeArguments ::=
    "[" commaSeparatedTypes "]"

typeParameters ::=
    "[" commaSeparatedTypeParameters "]"

functionType ::=
    typeParameters? "(" commaSeparatedTypes? ")" "->" type

expression ::=
    ifExpression
  | matchExpression
  | functionLiteral
  | block
  | pipelineExpression

pipelineExpression ::=
    binaryExpression { "|>" pipelineRhs }

pipelineRhs ::=
    callExpression
  | functionLiteral

binaryExpression ::=
    unaryExpression { binaryOperator unaryExpression }

unaryExpression ::=
    unaryOperator unaryExpression
  | callExpression

callExpression ::=
    fieldExpression { callSuffix }

callSuffix ::=
    "(" commaSeparatedExpressions? ")"

fieldExpression ::=
    primaryExpression { "." fieldName }

primaryExpression ::=
    literal
  | valueName
  | plainValueReference
  | qualifiedVariantExpression
  | structLiteral
  | builtinExpression
  | "(" expression ")"
  | "(" ")"

plainValueReference ::=
    "." valueName

literal ::=
    intLiteral
```

```
  | stringLiteral
  | boolLiteral

qualifiedVariantExpression ::=
    typeName typeArguments? "::" variantName variantArguments?

variantArguments ::=
    "(" commaSeparatedExpressions? ")"

structLiteral ::=
    typeName typeArguments? "::" "{" commaSeparatedStructLiteralFields "}"

structLiteralField ::=
    fieldName
  | fieldName ":" expression

builtinExpression ::=
    "builtin" "(" stringLiteral ")"

functionLiteral ::=
    "fn" typeParameters? "(" commaSeparatedFunctionLiteralParameters? ")"
functionReturnAnnotation? block

functionLiteralParameter ::=
    valueName
  | valueName space ":" space type

functionReturnAnnotation ::=
    "->" type

block ::=
    "{" localItem* expression "}"

localItem ::=
    localLetDeclaration
  | localOpenLetDeclaration
  | functionDeclaration
  | openDeclaration

ifExpression ::=
    "if" expression block "else" block

matchExpression ::=
    "match" expression "{" matchArm* "}"

matchArm ::=
    pattern "=>" expression

pattern ::=
    "_"
  | valueName
  | literal
  | qualifiedVariantPattern
  | structPattern

qualifiedVariantPattern ::=
    typeName "::" variantName patternArguments?

patternArguments ::=
    "(" commaSeparatedPatterns? ")"
```

```
structPattern ::=
    typeName "::" "{" commaSeparatedStructPatternFields "}"

structPatternField ::=
    fieldName
  | fieldName ":" pattern

binaryOperator ::=
    "+" | "-" | "*" | "/" | "%"
  | "==" | "!=" | "<" | "<=" | ">" | ">="
  | "&&" | "||"

unaryOperator ::=
    "-" | "!"

commaSeparatedTypes ::=
    type ("," type)* ","?

commaSeparatedTypeParameters ::=
    typeParameter ("," typeParameter)* ","?

commaSeparatedParameters ::=
    parameter ("," parameter)* ","?

commaSeparatedExpressions ::=
    expression ("," expression)* ","?

commaSeparatedFunctionLiteralParameters ::=
    functionLiteralParameter ("," functionLiteralParameter)* ","?

commaSeparatedStructLiteralFields ::=
    structLiteralField ("," structLiteralField)* ","?

commaSeparatedPatterns ::=
    pattern ("," pattern)* ","?

commaSeparatedStructPatternFields ::=
    structPatternField ("," structPatternField)* ","?

typeName ::=
    identifier

functionName ::=
    identifier

valueName ::=
    identifier

fieldName ::=
    identifier

variantName ::=
    identifier

typeParameter ::=
    identifier

space ::=
    whitespace+
```

The precedence and associativity of `binaryOperator`, `unaryOperator`, calls, field access, and pipeline expressions are defined in “Operators”.

2.4 Comments

```
comment ::=
    lineComment
  | blockComment

lineComment ::=
    "/" "/" lineCommentCharacter* lineTerminator?

lineCommentCharacter ::=
    any Unicode code point except U+000A or U+000D

blockComment ::=
    "/" "*" blockCommentCharacter* "*" "/"

blockCommentCharacter ::=
    any Unicode code point sequence that does not start "*/"
```

Comments are trivia. Comments do not appear in the syntactic grammar.

3 Type System

3.1 Notations

Notation	Meaning
T, U, R, F, P, Q	types
A	type variable
C	type constructor
S	struct type constructor
E	enum type constructor
e, c, f, b	expressions
a	match arm
x	value binder
i, j, k, n, m, q	indices and natural numbers
l	struct field name
v	enum variant name
Δ	type-constructor context
Θ	type-variable context
Γ	value typing context
D	custom type definition
$\Delta(C) = D$	visible custom type definition
$\text{struct}(A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m)$	struct definition
$\text{enum}\left(A_1, ..., A_n; v_{j\left(P_{j,1}, ..., P_{j,m_j}\right)}\right)$	enum definition
$\text{params}(D) = (A_1, ..., A_n)$	custom type parameters
σ	type substitution environment
$\sigma = [T_1/A_1, ..., T_n/A_n]$	substitution environment binding
$U[T_1/A_1, ..., T_n/A_n]$	direct type substitution
$U\sigma$	type substitution by environment
$\Delta\Theta \vdash T \;\; \text{type}$	well-formed type
$C[T_1, ..., T_n]$	nominal type application
$\forall A_1, ..., A_n.T$	universal type
$T \equiv U$	type equality
$\Gamma \vdash e : T$	expression typing
$\text{fields}(S[T_1, ..., T_n]) = (l_1 : Q_1, ..., l_m : Q_m)$	instantiated struct fields
$\text{payloads}(E[T_1, ..., T_n], v) = (Q_1, ..., Q_m)$	instantiated variant payloads
$\text{binders}(\Gamma x_1 : Q_1, ..., x_m : Q_m) = \Gamma'$	extended pattern-binder context
$\text{arm-type}(E[T_1, ..., T_n], R)$	typed enum match arm
$\frac{P}{Q}$	rule with premise P and conclusion Q
name	abstract syntax constructor

Table 1: Type system notations

3.2 Primitive Types

Lane2/Core v1 has four primitive types: `Unit`, `Bool`, `Int`, and `String`.

Primitive formation. Each primitive type is well-formed in every type environment.

$$\Delta\Theta \vdash \text{Unit} \;\; \text{type}$$

$$\Delta\Theta \vdash \text{Bool} \;\; \text{type}$$

$$\Delta\Theta \vdash \text{Int} \;\; \text{type}$$

$$\Delta\Theta \vdash \text{String} \;\; \text{type}$$

Unit introduction. The expression `()` introduces a `Unit` value.

```
() // unit literal
```

$$\Gamma \vdash () : \text{Unit}$$

Unit elimination. Lane2/Core v1 has no dedicated elimination form for `Unit`.

Bool introduction. Boolean literals introduce `Bool` values.

```
true  // boolean literal
false // boolean literal
```

$$\Gamma \vdash \text{true} : \text{Bool}$$

$$\Gamma \vdash \text{false} : \text{Bool}$$

Bool elimination. `if` eliminates a `Bool` value.

```
if c {
  e1
} else {
  e2
} // c : Bool, e1 : T, e2 : T
```

$$\frac{\Gamma \vdash c : \text{Bool} \quad \Gamma \vdash e_1 : T \quad \Gamma \vdash e_2 : T}{\Gamma \vdash \text{if}(c, e_1, e_2) : T}$$

Int introduction. Integer literals introduce `Int` values.

```
n // integer literal
```

$$\Gamma \vdash n : \text{Int}$$

Int elimination. Lane2/Core v1 has no built-in syntactic eliminator for `Int`. Integer operations are typed through required intrinsics and prelude operation values.

String introduction. ASCII string literals introduce `String` values.

```
"..." // ASCII string literal
```

$$\Gamma \vdash s : \text{String}$$

String elimination. Lane2/Core v1 has no built-in syntactic eliminator for `String`. String equality is typed through the prelude.

Primitive type equality. Primitive types are equal only to themselves.

$$\text{Unit} \equiv \text{Unit}$$

$$\text{Bool} \equiv \text{Bool}$$

$$\text{Int} \equiv \text{Int}$$

$$\text{String} \equiv \text{String}$$

3.3 Function Types

Function formation.

```
fn f(x1 : T1, ..., xn : Tn) -> R { e } // function definition
let f : (T1, ..., Tn) -> R = fn(x1, ..., xn) { e } // function literal binding
```

$$\frac{\Delta\Theta \vdash T_1 \ \text{type} \quad \ldots \quad \Delta\Theta \vdash T_n \ \text{type} \quad \Delta\Theta \vdash R \ \text{type}}{\Delta\Theta \vdash (T_1, \ldots, T_n) \to R \ \text{type}}$$

Function introduction.

```
fn(x1 : T1, ..., xn : Tn) -> R { e } // function literal
fn f(x1 : T1, ..., xn : Tn) -> R { e } // function definition
```

$$\frac{\Gamma, x_1 : T_1, \ldots, x_n : T_n \vdash e : R}{\Gamma \vdash \text{fn}((x_1 : T_1), \ldots, (x_n : T_n), R, e) : (T_1, \ldots, T_n) \to R}$$

Function elimination.

```
f(a1, ..., an) // function call
```

$$\frac{\Gamma \vdash f : (T_1, \ldots, T_n) \to R \quad \Gamma \vdash a_1 : T_1 \quad \ldots \quad \Gamma \vdash a_n : T_n}{\Gamma \vdash f(a_1, \ldots, a_n) : R}$$

Function type equality.

$$\frac{T_1 \equiv U_1 \quad \ldots \quad T_n \equiv U_n \quad R \equiv S}{(T_1, \ldots, T_n) \to R \equiv (U_1, \ldots, U_n) \to S}$$

Function types are uncurried. `(T1, T2) -> R` is not the same type object as `(T1) -> (T2) -> R`.

3.4 Generic Function Types

Generic function formation.

```
[A1, ..., An](T1, ..., Tm) -> R // generic function type
```

$$\frac{\Delta\Theta, A_1, ..., A_n \vdash (T_1, ..., T_m) \to R \ \ \text{type}}{\Delta\Theta \vdash \forall A_1, ..., A_n.(T_1, ..., T_m) \to R \ \ \text{type}}$$

Generic function introduction.

```
fn[A1, ..., An](x1 : T1, ..., xm : Tm) -> R { e } // generic function literal
fn[A1, ..., An] f(x1 : T1, ..., xm : Tm) -> R { e } // generic function definition
```

A generic function literal or named generic function introduces a generic function value.

Generic function elimination.

Generic function calls instantiate type parameters at the use site when the use site is unambiguous.

```
f(a1, ..., am) // generic function call with inferred type arguments
```

Generic function type equality.

$$\frac{(T_1, ..., T_m) \to R \equiv (U_1, ..., U_m) \to S}{\forall A_1, ..., A_n.(T_1, ..., T_m) \to R \equiv \forall A_1, ..., A_n.(U_1, ..., U_m) \to S}$$

Generic function type equality compares the number of type parameters and the function types under corresponding bound type parameters.

3.5 Nominal Custom Types

Struct and enum declarations introduce nominal custom type definitions in Δ.

```
struct S[A1, ..., An] {
  l1 : F1
  ...
  lm : Fm
} // struct definition
```

```
enum E[A1, ..., An] {
  v1(P11, ..., P1m1)
  ...
  vj(Pj1, ..., Pjmj)
} // enum definition
```

The type-constructor context stores the full declaration shape, not only arity.

$$\Delta(S) = \text{struct}(A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m)$$

$$\Delta(E) = \text{enum}\left(A_1, ..., A_n; v_{j\left(P_{j,1}, ..., P_{j,m_j}\right)}\right)$$

Top-level custom type declarations are checked in two phases. First, all top-level struct and enum constructors are collected into Δ with their type parameters, field names, variant names, and declared member type expressions. Second, every field type and variant payload type is checked under the collected Δ and the declaration's type parameters. This permits mutually recursive custom types.

$$\frac{\Delta(S) = \text{struct}(A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m) \quad \forall i. \Delta A_1, ..., A_n \vdash F_i \ \ \text{type}}{\Delta \vdash S \ \ \text{declaration}}$$

$$\frac{\Delta(E) = \text{enum}\left(A_1, ..., A_n; v_{j\left(P_{j,1}, ..., P_{j,m_j}\right)}\right) \quad \forall j, k. \Delta A_1, ..., A_n \vdash P_{j,k} \ \ \text{type}}{\Delta \vdash E \ \ \text{declaration}}$$

Nominal formation. A nominal type application is well-formed when its custom type constructor is visible, the number of type arguments matches the declaration's type parameters, and every type argument is well-formed.

$$\frac{\Delta(C) = D \quad \text{params}(D) = (A_1, ..., A_n) \quad \Delta\Theta \vdash T_1 \ \ \text{type} \quad ... \quad \Delta\Theta \vdash T_n \ \ \text{type}}{\Delta\Theta \vdash C[T_1, ..., T_n] \ \ \text{type}}$$

Nominal type equality. Nominal type equality compares constructor identity and then compares type arguments pairwise. There is no equality rule whose conclusion relates different nominal constructors.

$$\frac{T_1 \equiv U_1 \quad ... \quad T_n \equiv U_n}{C[T_1, ..., T_n] \equiv C[U_1, ..., U_n]}$$

3.6 Struct Types

Struct introduction. A qualified struct literal introduces a struct value. It must provide every declared field exactly once. Source field order is not significant; the rule below uses declaration order.

```
S[T1, ..., Tn]::{ l1: e1, ..., lm: em } // qualified struct literal
```

$$\frac{\Delta(S) = \text{struct}(A_1, ..., A_n; l_1 : F_1, ..., l_m : F_m) \quad \sigma = [T_1/A_1, ..., T_n/A_n]}{\text{fields}(S[T_1, ..., T_n]) = (l_1 : F_1\sigma, ..., l_m : F_m\sigma)}$$

$$\frac{\text{fields}(S[T_1, ..., T_n]) = (l_1 : Q_1, ..., l_m : Q_m) \quad \Gamma \vdash e_1 : Q_1 \quad ... \quad \Gamma \vdash e_m : Q_m}{\Gamma \vdash \text{struct-lit}(S[T_1, ..., T_n], l_1 = e_1, ..., l_m = e_m) : S[T_1, ..., T_n]}$$

Struct field elimination. Field access eliminates a struct value by selecting a declared field type after substituting the struct type arguments.

```
e.l // field access
```

$$\frac{\Gamma \vdash e : S[T_1, ..., T_n] \quad \text{fields}(S[T_1, ..., T_n]) = (..., l : Q, ...)}{\Gamma \vdash e.l : Q}$$

Struct values are also eliminated by struct patterns and exposed by `open`; their detailed static rules are specified in "Pattern Matching" and "Structs, Enums, and Open".

3.7 Enum Types

Enum variant introduction. A qualified enum variant expression introduces an enum value. The payload expressions must match the declared variant payload types after substituting the enum type arguments.

```
E[T1, ..., Tn]::v(e1, ..., em) // qualified variant expression
v(e1, ..., em) // unqualified variant expression after unambiguous resolution
```

$$\frac{\Delta(E) = \text{enum}(A_1, ..., A_n; ..., v(P_1, ..., P_m), ...)}{\text{payloads}(E[T_1, ..., T_n], v) = \left(P_1[T_1/A_1, ..., T_n/A_n], ..., P_{m[T_1/A_1, ..., T_n/A_n]}\right)}$$

$$\frac{\text{payloads}(E[T_1, ..., T_n], v) = (Q_1, ..., Q_m) \quad \Gamma \vdash e_1 : Q_1 \quad ... \quad \Gamma \vdash e_m : Q_m}{\Gamma \vdash \text{variant}(E[T_1, ..., T_n], v, e_1, ..., e_m) : E[T_1, ..., T_n]}$$

An unqualified enum variant expression is typed by the same rule after name resolution has selected exactly one visible enum variant.

Enum match elimination. A match expression eliminates an enum value. For an enum scrutinee type, every declared variant must be covered.

```
match e {
  E::v1(x11, ..., x1m1) => b1
  ...
  E::vq(xq1, ..., xqmq) => bq
}
```

$$\frac{\text{payloads}(E[T_1, ..., T_n], v) = (Q_1, ..., Q_m)}{\text{binders}(\Gamma x_1 : Q_1, ..., x_m : Q_m) = \Gamma'}$$

$$\frac{\text{payloads}(E[T_1, ..., T_n], v) = (Q_1, ..., Q_m) \quad \text{binders}(\Gamma x_1 : Q_1, ..., x_m : Q_m) = \Gamma' \quad \Gamma' \vdash b : R}{\Gamma \vdash \text{arm}(v(x_1, ..., x_m) \Rightarrow b) : \text{arm-type}(E[T_1, ..., T_n], R)}$$

$$\frac{\Gamma \vdash e : E[T_1, ..., T_n] \quad \Delta(E) = \text{enum}\left(A_1, ..., A_n; v_1, ..., v_q\right) \quad \Gamma \vdash a_j : \text{arm-type}(E[T_1, ..., T_n], R)}{\Gamma \vdash \text{match}\left(e; a_1, ..., a_q\right) : R}$$

Pattern syntax and exhaustiveness diagnostics are specified in “Pattern Matching”.

4 Type Inference

Lane2 permits local type inference only at syntactic positions where the omitted type is determined locally.

A binding’s type is not determined from later uses of the bound name.

Local `let` bindings may omit type annotations only when the initializer has a type without using later references to the bound name.

Function literals may omit parameter types only when the immediately surrounding context provides a function type. Otherwise, every value parameter of a function literal must have an explicit type annotation.

Generic function literals without an immediately surrounding generic function type must explicitly declare type parameters and explicitly annotate value parameters.

Generic function calls and generic data constructors may instantiate type parameters at the use site when the use site is unambiguous.

5 Builtins

`builtin("...")` is an unsafe expression.

The checker does not interpret intrinsic strings. Instead, `builtin` receives its type from direct expected context:

```
fn int_add(a : Int, b : Int) -> Int {
  builtin("%i64_add")
}
```

This is valid because the function return type provides the expected type `Int` for the body expression.

This is invalid:

```
let x = builtin("%anything")
```

There is no direct expected type.

Incorrect builtin use can produce undefined behavior. Lane2′s type safety guarantee applies only to programs that do not misuse `builtin`.

5.1 Required Intrinsics

A conforming Lane2/Core v1 implementation provides these portable intrinsic names:

Intrinsic	Expected type
`%i64_add`	`(Int, Int) -> Int`
`%i64_sub`	`(Int, Int) -> Int`
`%i64_mul`	`(Int, Int) -> Int`
`%i64_div`	`(Int, Int) -> Int`
`%i64_rem`	`(Int, Int) -> Int`
`%i64_neg`	`(Int) -> Int`
`%i64_equal`	`(Int, Int) -> Bool`
`%i64_less`	`(Int, Int) -> Bool`
`%string_equal`	`(String, String) -> Bool`

Table 2: Required intrinsic names and expected types

Other intrinsic names are implementation-defined unsafe builtins.

`%bool_and`, `%bool_or`, `%bool_not`, and `%bool_equal` are not required intrinsics. The standard prelude defines boolean operations as ordinary Lane2 functions and open bindings using `if`; `&&` and `||` supply their right operands as thunks.

6 Prelude

The standard prelude is implementation-supplied Lane2 source checked before user code. It is not a module system.

Prelude declarations provide standard operation structs, primitive wrappers around required intrinsics, derived primitive operations written in Lane2, and prelude-provided open bindings that populate the initial preopen namespace.

Prelude entries are ordinary Lane2 values and types. Operation structs are API conventions that group ordinary operation names.

6.1 Standard Operation Structs

Arithmetic operations:

```
struct Add[T] {
  op_add : (T, T) -> T
}
```

```
struct Sub[T] {
  op_sub : (T, T) -> T
}

struct Mul[T] {
  op_mul : (T, T) -> T
}

struct Div[T] {
  op_div : (T, T) -> T
}

struct Rem[T] {
  op_rem : (T, T) -> T
}

struct Neg[T] {
  op_neg : (T) -> T
}
```

Boolean operations:

```
struct And {
  op_and : (Bool, () -> Bool) -> Bool
}

struct Or {
  op_or : (Bool, () -> Bool) -> Bool
}

struct Not {
  op_not : (Bool) -> Bool
}
```

Equality and ordering operations:

```
struct Equal[T] {
  op_equal : (T, T) -> Bool
  op_not_equal : (T, T) -> Bool
}

struct Compare[T] {
  equal_impl : Equal[T]
  open equal_impl
  op_less : (T, T) -> Bool
  op_less_eq : (T, T) -> Bool
  op_greater : (T, T) -> Bool
  op_greater_eq : (T, T) -> Bool
}
```

Operation laws are API conventions, not compiler-checked rules.

6.2 Standard Preopen Bindings

The prelude may populate the initial preopen namespace with prelude-provided open bindings. For example, primitive arithmetic and boolean operations can be exposed as follows:

```
let open int_add_ops : Add[Int] = Add::{ op_add: int_add }
let open int_sub_ops : Sub[Int] = Sub::{ op_sub: int_sub }
let open int_mul_ops : Mul[Int] = Mul::{ op_mul: int_mul }
```

```
let open int_div_ops : Div[Int] = Div::{ op_div: int_div }
let open int_rem_ops : Rem[Int] = Rem::{ op_rem: int_rem }
let open int_neg_ops : Neg[Int] = Neg::{ op_neg: int_neg }

let int_equal_ops : Equal[Int] = make_equal(int_equal)
let open int_compare_ops : Compare[Int] = make_compare(int_equal_ops, int_less)

let open bool_and_ops : And = And::{ op_and: bool_and }
let open bool_or_ops : Or = Or::{ op_or: bool_or }
let open bool_not_ops : Not = Not::{ op_not: bool_not }
let open bool_equal_ops : Equal[Bool] = Equal::{
  op_equal: fn(a : Bool, b : Bool) {
    if a {
      b
    } else {
      bool_not(b)
    }
  },
  op_not_equal: fn(a : Bool, b : Bool) {
    if a {
      bool_not(b)
    } else {
      b
    }
  },
}

let open string_equal_ops : Equal[String] = make_equal(string_equal)
```

Boolean conjunction, disjunction, negation, and equality do not require boolean intrinsics; they can be defined with ordinary `if` expressions in the prelude.

7 Declarations

Declarations introduce types, functions, values, and open scope extensions.

7.1 Top-Level Declarations

Top-level declarations are described by the `topLevelDefinition` grammar in “Syntax and Grammar”.

Top-level forms include struct declarations, enum declarations, named function declarations, typed value declarations, open bindings, and open declarations.

7.2 Struct Declarations

Struct declarations use the grammar in “Structs, Enums, and Open”.

7.3 Enum Declarations

Enum declarations use the grammar in “Structs, Enums, and Open”.

7.4 Function Declarations

Functions are uncurried. There is no automatic currying or partial application.

Function definitions use block bodies:

```
fn add(a : Int, b : Int) -> Int {
  a + b
}
```

There is no `= expr` function body form.

Function result types use `->`, not `:`.

All named functions must state every parameter type and the result type.

Generic named functions place type parameters after `fn` and before the name:

```
fn[A] id(value : A) -> A {
  value
}
```

Function parameters are positional in v1. Labeled function parameters are not supported.

Function literals produce function values:

```
let f : (Int, Int) -> Int = fn(a, b) {
  a + b
}
```

Generic function literals place type parameters after `fn`:

```
let id = fn[A](value : A) {
  value
}
```

7.5 Let Declarations

Named top-level `let` declarations must include type annotations.

Local `let` declarations may omit type annotations when their initializer can synthesize a type by local type inference.

An open binding has the form `let open name ... = expression`. It is syntax sugar for a value declaration immediately followed by `open name`.

Top-level open bindings must include type annotations because all top-level value declarations must include type annotations:

```
let open int_ops : Add[Int] = Add::{ op_add: int_add }
```

Local open bindings may omit type annotations when their initializer can synthesize a unique struct type:

```
{
  let open ops = Add::{ op_add: int_add }
  1 + 2
}
```

Open bindings are not forward-visible and are not recursively open.

8 Scopes and Bindings

Top-level type and function definitions form recursive definition groups.

Top-level `struct`, `enum`, and `fn` definitions may refer to each other regardless of textual order.

Top-level value definitions and top-level `open` declarations follow ordered value scope:

- a top-level `let` initializer may refer only to values already available at that declaration point;
- a top-level `open` may open only a value already available at that declaration point;
- a top-level `let open` defines its value and opens it only from that declaration point forward;
- a top-level function body may refer to any top-level value or function, even one declared later.

Example:

```
fn read_x() -> Int {
  x
}

let x : Int = 1
```

This is valid because the function body sees the complete top-level environment.

```
let y : Int = x
let x : Int = 1
```

This is invalid because top-level value initializers follow ordered value scope.

8.1 Namespaces

Lane2 has separate type and value namespaces.

Type and value names may use the same spelling:

```
enum Option[A] {
  none
  some(A)
}

let Option : Int = 42
```

Syntax of the form `Type::member` resolves `Type` in the type namespace.

```
let x : Option[Int] = Option::some(1)
```

The `Option` on the left of `::` denotes the enum type, not the value named `Option`.

8.2 Local Bindings

Local `let` bindings are sequential. A local binding is visible only to later items and the final expression in the same block.

Local `let` bindings may omit type annotations when their initializer can synthesize a type by local type inference:

```
let x = 1
```

Local named functions are also sequential. They may call themselves, but local mutually recursive groups and forward references are not supported:

```
fn f(n : Int) -> Int {
  fn loop(x : Int) -> Int {
    if x == 0 {
      0
    } else {
      loop(x - 1)
    }
  }
  loop(n)
}
```

Local value names may shadow earlier local value names.

Ordinary value bindings in the same scope must have distinct names. Function parameters, local lets, top-level values, local functions, and pattern binders are ordinary value bindings.

8.3 Open Scope Extensions

`open value` opens a struct value.

The operand of `open` must be a visible value name. It cannot be an arbitrary expression:

```
open ops
```

This is invalid:

```
open make_ops(10)
```

The opened value must have a struct type after checking the target binding. The type can come from a type annotation or from an already checked earlier binding:

```
{
  let ops = Add::{ op_add: int_add }
  open ops
  1 + 2
}
```

If the target is missing or is not a struct value, the `open` declaration is invalid at its declaration point.

An open scope extension exposes the struct field values as unqualified candidates from the declaration point to the end of the current lexical layer.

At top level, `open` extends to the end of the file:

```
open int_add_ops

fn add_one(x : Int) -> Int {
  x + 1
}
```

Inside blocks, `open` is a local item and must appear before the final expression:

```
{
  open int_add_ops
  x + y
}
```

Plain value bindings and open scope extensions use separate lexical shadowing. Plain value bindings shadow only other plain value bindings. Open scope extensions shadow only outer open scope extensions and preopen exposures.

An unqualified value reference combines the nearest plain binding candidate with the candidates exposed by the nearest open scope layer. If multiple candidates remain applicable after local type checking, the reference is ambiguous.

8.4 Preopen

The preopen namespace is a default-open scope layer prepared by the prelude before user code is checked.

Preopen exposes field values, not generated accessors. Each preopen exposure originates from a prelude-provided open binding whose value has a struct type.

Prelude-provided preopen exposures are visible throughout user code.

User-defined top-level open bindings contribute to the top-level open layer from their declaration point to the end of the top-level scope.

Preopen exposure name repetition is not an error at the declaration point. Repeated names form candidate sets and are reported only when a use cannot select exactly one candidate.

Top-level `open` contributes candidates to the top-level lexical layer while preserving ordered top-level value scope. Local `open` contributes candidates to the local lexical layer and shadows outer open layers, including preopen.

9 Expressions

Expressions compute values. Lane2 v1 is expression-oriented and has no expression statements.

9.1 Blocks

A block expression contains local items followed by one final expression.

```
{
  let x = 1
  fn double(n : Int) -> Int {
    n + n
  }
  double(x)
}
```

Allowed local items:

- `let`,
- named `fn`,
- `open`.

Local type definitions are not supported in v1.

A block does not contain expression statements. This is invalid:

```
{
  f(x)
  g(y)
}
```

An empty block is invalid. Write `()` explicitly when a `Unit` value is required.

9.2 Conditional Expressions

`if` is an expression:

```
if condition {
  then_value
} else {
  else_value
}
```

The `else` branch is mandatory.

Both branches must have the same type.

9.3 Calls, Field Access, and Pipeline

An unqualified value reference may denote a candidate set. Candidate selection uses direct local typing information:

- a direct expected type, including expected result types for function bodies and branch expressions;
- function call argument count and argument checking against candidate parameter types;
- generic instantiation determined by the same direct local typing information.

Lane2 does not choose a default candidate. If more than one candidate remains applicable, the reference is ambiguous. Diagnostics for ambiguous candidates should list source-level disambiguation paths such as `.name` for an ordinary binding and `owner.field` for an opened field.

A plain value reference begins with `.` and resolves only ordinary lexical value bindings:

```
.op_add
```

Plain value references still follow ordinary lexical shadowing among ordinary bindings, but exclude open and preopen exposures.

Function call:

```
f(x, y)
```

Field access:

```
ops.add
```

The base of field access must resolve and check as a unique value before field selection. Field access does not search through an unresolved candidate set for a base value.

Field access followed by call:

```
ops.add(x, y)
```

This is not method call syntax. Lane2 v1 has no method receiver lookup.

Pipeline syntax is supported:

```
value |> f(a, b)
```

It rewrites to:

```
f(value, a, b)
```

The right-hand side of `|>` must be a call or function literal.

The following are invalid:

```
value |> f
value |> f(_, y)
```

Placeholders are not supported.

10 Structs, Enums, and Open

Struct literals are qualified:

```
let p : Point = Point::{ x: 1, y: 2 }
```

Struct field punning is allowed:

```
let p : Point = Point::{ x, y }
```

This is equivalent to:

```
let p : Point = Point::{ x: x, y: y }
```

Struct literals must provide every field exactly once.

The following are not supported:

- anonymous record literals,
- default fields,
- spread,
- copy update,
- field update.

Fields are accessed with dot syntax:

```
p.x
```

V1 has no field visibility modifiers. Every struct field is accessible wherever the struct value is accessible.

10.1 Enums

Enum variants are scoped to their enum.

Variant names may be lowercase or uppercase. Capitalization has no semantic role.

Payloadless variants are values and are written without call parentheses:

```
let x : Option[Int] = Option::none
```

This is invalid:

```
Option::none()
```

Variant payloads are positional:

```
enum Expr {
  int(Int)
  add(Expr, Expr)
}
```

Labeled variant payloads are not part of v1. Named product data must be represented with a struct:

```
struct Node[A] {
  left : Tree[A]
  value : A
  right : Tree[A]
}

enum Tree[A] {
  leaf(A)
  node(Node[A])
}
```

In expressions, unqualified variants are allowed when unambiguous:

```
let x : Option[Int] = some(1)
```

If more than one visible enum has a variant named `some`, the unqualified expression is invalid unless another directly local rule disambiguates it. A qualified variant is always allowed:

```
let x : Option[Int] = Option::some(1)
```

In patterns, variants must always be qualified.

10.2 Openable Struct Values

Only struct values can be opened.

Opening a struct value exposes its field values as unqualified bindings according to the scope rules in “Scopes and Bindings”.

Enum values, function values, primitive values, and arbitrary expressions are not openable.

10.3 Struct Field Forwarding

A struct declaration may contain `open field` entries:

```
struct Compare[T] {
  equal_impl : Equal[T]
  open equal_impl
  op_less : (T, T) -> Bool
  op_less_eq : (T, T) -> Bool
  op_greater : (T, T) -> Bool
  op_greater_eq : (T, T) -> Bool
}
```

When a value of this struct type is opened, forwarded fields are exposed too. Direct fields and forwarded fields from the same opened struct value contribute to the same open exposure layer.

Repeated exposed names are not conflicts at the open declaration point. Ambiguity is reported only at use sites when candidate selection cannot choose exactly one value.

Struct field forwarding affects only what is exposed by opening the containing struct value. It does not open the field inside ordinary function bodies.

11 Pattern Matching

`match` is an expression:

```
match value {
  Option::none => fallback
  Option::some(x) => x
}
```

Match arms use `pattern => expression`.

Every match must be exhaustive.

V1 patterns:

- wildcard pattern: `_`,
- variable binding pattern: `name`,
- literal pattern,
- qualified enum variant pattern,
- qualified struct pattern.

Enum variants in patterns must be qualified:

```
Option::some(x)
Option::none
```

Bare identifiers in patterns are always variable bindings:

```
match value {
  x => x
}
```

Struct patterns use qualified syntax and must list all fields:

```
match p {
  Point::{ x, y } => x + y
}
```

Struct pattern punning is allowed. Explicit renaming is allowed:

```
match p {
  Point::{ x: a, y: b } => a + b
}
```

Rest patterns, spread patterns, guards, or-patterns, as-patterns, and `is` pattern expressions are not supported in v1.

12 Operators

Operators are aliases for fixed ordinary operation names.

There is no trait, typeclass, or instance search. An operator is available only when the corresponding operation name resolves to a suitable value through ordinary value lookup and open candidate selection.

Primitive operators are not special-cased. For example, `1 + 2` resolves through the ordinary name `op_add`; that name can be supplied by a normal binding or by opening a struct value with an `op_add` field.

Recognized operator mappings:

Operator	Operation name
`+`	`op_add`
`-`	`op_sub`
`*`	`op_mul`
`/`	`op_div`
`%`	`op_rem`
unary `-`	`op_neg`
`==`	`op_equal`
`!=`	`op_not_equal`
`<`	`op_less`
`<=`	`op_less_eq`
`>`	`op_greater`
`>=`	`op_greater_eq`
`&&`	`op_and` with a thunked right operand
`||`	`op_or` with a thunked right operand
`!`	`op_not`

Table 3: Recognized operator mappings

Operation names such as `op_add` are ordinary value names, not reserved words. Lane2 v1 does not allow user-defined operator tokens or user-defined operator mappings.

`&&` and `||` are short-circuit boolean operators. They resolve through `op_and` and `op_or`, but the right operand is passed as a zero-argument function instead of being evaluated before the operation call.

`a && b` desugars to a call equivalent to `op_and(a, fn() { b })` after resolving an available `op_and` operation. `a || b` follows the same rule with `op_or`. Both operators are defined only for `Bool` in v1.

Ordinary calls to `op_and` and `op_or` are strict. Only the `&&` and `||` operator syntaxes thunk the right operand.

Concrete expression precedence, associativity, and unary/binary disambiguation follow the MoonBit parser reference where Lane2 has not deliberately removed a feature.

13 Evaluation

Lane2 v1 is pure and strict.

- Evaluating a safe expression depends only on its inputs and produces a value.
- Function arguments are evaluated before the function body runs.
- `if` and `match` evaluate only the selected branch.
- There is no mutable binding, mutable field, assignment, field update, or implicit statement sequencing.
- IO and other effects are not part of v1 core semantics.

The `Unit` value is written explicitly as `()`.

Empty blocks are invalid. A block must contain exactly one final expression after any local items.

Function calls evaluate all arguments before entering the function body. The only v1 operator forms that delay a subexpression are `&&` and `||`, which pass the right operand as a thunk to the resolved `op_and` or `op_or` value.

14 Undefined Behavior

Incorrect builtin use can produce undefined behavior. Lane2′s type safety guarantee applies only to programs that do not misuse `builtin`.

Invalid integer arithmetic is undefined behavior in v1. This includes signed 64-bit overflow, division by zero, remainder by zero, `MIN_INT / -1`, and negating `MIN_INT`.

15 Conformance

The words “must”, “must not”, “may”, and “should” are normative in this document.

Text marked as rationale, examples, or notes is informative unless it explicitly uses normative language.

An implementation conforms to this draft if it accepts all valid programs described here, rejects invalid programs described here, and gives the specified typing and evaluation behavior for safe programs.

Programs that misuse `builtin` are outside Lane2′s safety guarantee.

16 Appendix

16.1 Deliberately Omitted From V1

V1 omits:

- mutation and assignment,
- field update and copy update,
- modules and imports,
- local type definitions,
- labeled function parameters,
- labeled enum payloads,
- type aliases,
- traits, typeclasses, interfaces,
- method syntax,
- tuple types,
- collections,
- pattern guards,
- or-patterns,
- as-patterns,

- `is` pattern expressions,
- placeholder pipeline.